
ARTICLE

Optimized Term Importance Framework for Automated Feature Derivation in Complex and High-Dimensional Big Data

Yanglem Loijing Khomba Khuman^{1,*} and Iyyappan Moorthi²

¹Assistant Professor, Department of Computer Science, Naorem Birahari College, Khundrakpam, Manipur, India.

²Assistant Professor, College of Information Technology, Ahlia University, Bahrain.

*Corresponding Author: Yanglem Loijing Khomba Khuman. Email: loijingyanglem@gmail.com

Received: 14 June 2024; Accepted: 11 September 2024; Published: 30 December 2024

ABSTRACT: In the recent trends of the Big-data, A large volume of data, mostly unstructured, is produced in a variety of forms such as Text, Images, Audio, and video. Making use of these data in an effective manner is a tedious and challenging task. Feature Extraction methods are used widely for extracting meaningful information from these large data sources. Data dimensionality is a critical issue when Data mining algorithms are applied to large data. Although many studies have been conducted in the Feature Engineering of unstructured data to address the dimensional complexity, most of the methods suffer from pitfalls in one or the other metric. This paper proposes an automated Feature Extraction method based on a weighted TF-IDF model using the unigram vector space method for a large text corpus. The document similarity features are calculated using the cosine similarity, and Document Clustering is done for grouping the similar features from the corpus together. Experimental analysis proves that the proposed methodology outperforms the other state-of-the-art embedded methods for text feature extraction.

KEYWORDS: Big data, Feature Extraction, Dimensionality, TF-IDF model, Topic modelling, and LDA.

1 Introduction

The recent advancement in technology and storage space has made the accumulation of data grow exponentially with time in the past decade. The Volume, Variety, and Velocity of large data have drastically improved the computational abilities of the systems. The prediction of IDC says that unstructured data will contribute to almost 95% percent of the total data available in the next 5 years [1]. The characteristics that make the retrieval of useful information from unstructured data challenging are first it takes up different formats. Second, these are unlike traditional databases that have a proper schema definition. Third, these do not have a standard, and finally, it is generated from various sources, from basic forms to cloud sensors and IoT devices [2-6]. Owing to the reason that these data are too complex and of large volume, extracting useful information from these has always been a difficult task. Feature Engineering is a process of extracting useful information from large unstructured data. This information, extracted from a large corpus, is used for the data analysis. An efficient and correct FE method can be vital in the data analysis that contributes a lot to any business's success. A handful of research has been conducted on this issue in the recent decade.

Text form of data is the most commonly found unstructured data. These normally consisted of documents that had words, paragraphs, and sentences, and had a free flow without any written



semantics. These data are normally found to be inconsistent, noisy, and unstructured, which makes the process of extracting features a tedious task. Feature Engineering is perhaps known as the tool for creating efficient data models [7]. This is more important in the case of handling text data, as the task here would be to convert the freely flowing text into numeric data that a system or algorithm understands.

The high-dimensionality nature of text data will end up producing many features that are least important and thus increase the computational cost and memory required. Feature Extraction is a kind of dimension reduction technique that has been proven as an effective way to handle high-dimensional data.

Traditionally, analysts used to create features using a manual process from domain or/business knowledge. Often, it is called handcrafted feature engineering, which is more complex, time-consuming, and unbiased [8]. Feature Extraction in text data is usually performed using three methods. i. The filter methods are mostly used in the pre-processing phase. Here, the selection of a particular feature is not dependent on any one machine learning algorithm. The features are extracted based on the weighted scores obtained from different statistical methods for the correlation of the output variable. ii. Wrapper methods are those where a subset of the features is then used for training the model [9]. iii. Embedded methods are those that combine the qualities of both (i) and (ii). These are implemented using the algorithms that have in-built methods for feature extraction [10].

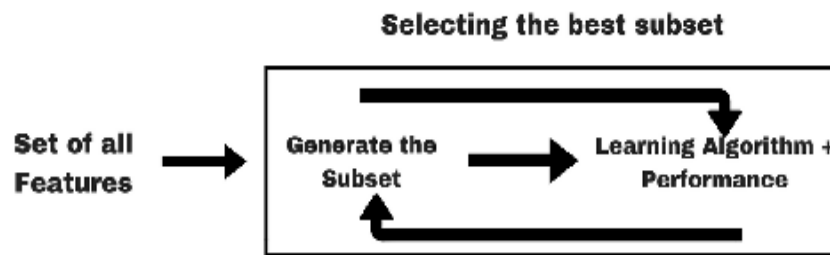


Figure 1: Embedded Feature Extraction Model

This research is carried out using an embedded technique, namely the Weighted-Term Frequency-Inverse Document Frequency (TF-IDF) with a weighted vector space model for automated extraction of features from text documents. Further, document similarity features are calculated using the cosine similarity functions and comparing the pairwise document similarity based on the TF-IDF feature vectors. Document Clustering is done to leverage the unsupervised hierarchical clustering algorithm for grouping similar documents from the corpus together by leveraging the document similarity features extracted. The rest of the paper is organized as follows. Section II gives the gist of available methods in a similar domain. Section III explains the feature extraction problem from text data. Section IV explains the proposed methodology in detail, and Section V discusses the experimental setup and data description. Section VI interprets the results obtained and the comparative analysis. Section VII concludes the paper with closing remarks and limitations, and scope for future research.

2 Related Works

Jie Chen et al (2016) introduced the adaptive weight concept in the traditional TF-IDF method. This was capable of determining the position weight dynamically based on the position of a particular word. The Vector Space model was introduced and tested in the context of Chinese-based document

clustering. The results showed an increase of 12% in performance over that of the traditional TF-IDF. Although it performed better, this method had a severe drawback in terms of log loss and hinge losses [11].

Ghayda et al (2017) proposed a weighted method that is based on the statistical prediction of the words' importance in a SMS categorization issue. This method was intended to classify the SMS in mobile into a predefined set of classes, such as greeting, relation, sales, etc All the messages were converted into an unstructured text corpus. Once the pre-processing is over, the Vector model is prepared, and the weights are assigned for all the terms. This method was based on the categorization of texts, and the experiments proved a great accuracy in the classification. One of the major issues to be addressed is the time complexity when more weights are added to the classifiers [12].

Amandeep Singh et al (2018) introduced the Supervised ML approach as a combination of a count vector and the traditional TF-IDF, which is based on the Chi-Square for the Feature Extraction. The proposed method combines the different N-gram features that signify various aspects of the sentiment inside the text data. The proposed method performed better than other models by the use of a count-based TF-IDF. The SVM classifier is considered to be the better one after the experimental evaluation with the proposed method [13].

Z.ZHU et al (2019) proposed a re-defined term frequency and the Inverse document frequency for finding the hot topics, which is based on the time distribution and attention of the users. A method was also put forth for generating new terms and also to combine terms that are split using the Chinese segmentation method. The hot news is then extracted using the hot terms, and K-means clustering is done for grouping the realization of the hot topics in the news. The experiments prove that the results obtained through the proposed refined TF-IDF are effective for finding the hot topics [14].

YAHAV et al (2019) introduced the bias between the participants in the comments mining from social media data. It was found that the content which was extracted from the discourse often has high correlation, which results in structures having dependency in-between recordings in the study and thus introduces a statistical bias. Ignorance of this will lead to a non-robust analysis, which can drive the decision makers to a wrong conclusion. The traditional TF-IDF is adjusted here to handle the bias. The experiments were carried out on Facebook data that covers various domains, including news, finance, sports, and other entertainment [15].

Kim, S et al (2019) proposed research on a classification technique that performs clustering of the research papers into a meaningful group for easy understanding, as to which research articles are likely to focus on similar subjects. The method proposed extracts the keywords from the abstracts of the paper using the LDA scheme. The K-Means clustering is performed for the classification of the entire papers into a group of papers that focus on similar subjects based on the TF-IDF value of all the papers [16].

Y. Li et al (2015) presented an innovative solution for the relevant feature identification. It identifies both the positive and the negative patterns in the free-flowing text as next-level features and then deploys them into lower features. Classification of the terms into suitable categories is also done based on the weights of the specificity and the distribution patterns. The experiments show that the proposed method significantly performs better than the other pattern-based methods [17].

3 Proposed Methodology

The overall architecture can be viewed in four phases, namely the preprocessing phase, the Feature Extraction phase, the Clustering phase, and the summarization phase. Figure 2 shows the architecture of the proposed methodology.

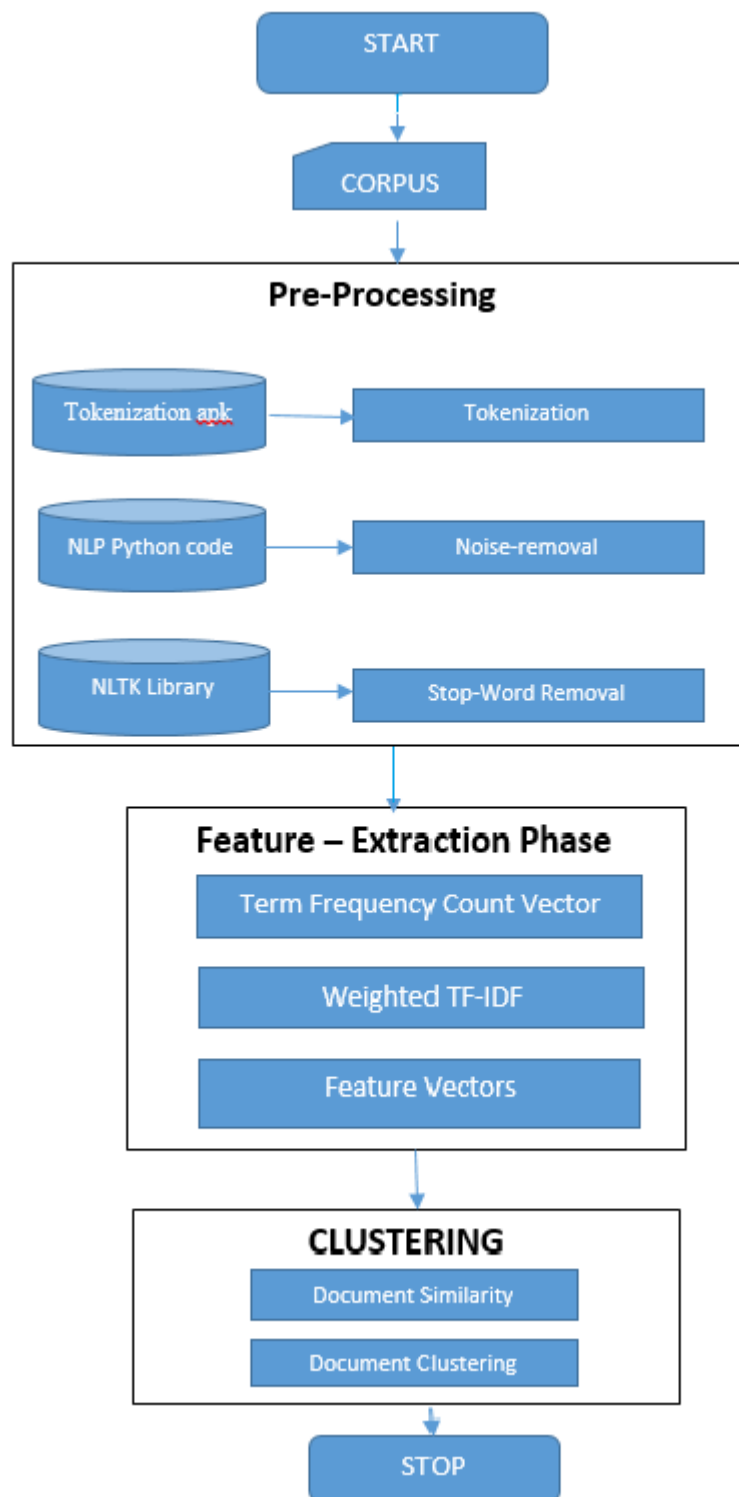


Figure 2: Architecture of the proposed methodology

The proposed architecture is explained on the basis of the corpus considered for the experiment. The text corpus is taken from the US Departmental score card data of size 256 Mb of text content with

2483 text documents. For a better understanding of the proposed methodology. A sample portion, as shown in Figure 3 from the corpus, is considered.

	Document	Category
1	Alex was the topper in the computer paper of class 2016	Performance
2	Tony have got suspension for rude behavior in current term	Conduct
3	Smiley had pneumonia during last term exams	Absentism
4	Alex was born on 19 th January 1991 and his parents are no more	personal
5	He got the best outgoing student award of the year	Conduct
6	She has failed in many of the subjects	Performance
7	He is a transgender and his height was 165 Cm	personal
8	All the medical leave submitted are pre-fixed with declared holidays	Absentism

Figure 3: Sample Corpus

3.1 Preprocessing

Data preprocessing is carried out in three methods, namely tokenization, normalization, and stop word removal.

3.1.1 Tokenization

The process of chopping the given sentence into smaller parts (tokens) is known as tokenization [18]. By and large, the given crude content is tokenized dependent on a lot of delimiters (for the most part whitespaces). Tokenization is utilized in tasks, for example, spell-checking, preparing text, recognizing grammatical forms, sentence discovery, document classification of reports, and so forth. Tokenizing utilizing Open NLP.

Step 1 – Instantiating the respective class.

Step 2 – Tokenize the sentences.

Step 3 – Extract the tokens and save them in the repository that was created manually.

We have used the tokenization apk available in the Open NLP package for the process. In the data that were considered for our findings.

3.1.2 Noise Removal

Noise removal is the next pre-processing step to be performed. The text file does have a lot of noise, such as punctuation, short forms, and linguistic extensions. These are eliminated by,

- Removing the headers of the files and footers.
- Removing the markup, such as HTML and XML.
- Extracting interesting information from different formats, Ex. JSON or from inside the DBs.

3.1.3 Stop-word Removal

Once the cleaning is over, words like “is, am, are, an”, which are considered the stop words, are to be removed. These are done using the NLTK pre-defined stop words [19]. The pre-processing pipeline and implemented in the corpus, which yielded the results as depicted in Figure 4.

```

Norm_Corpus = Normalize_corpus (Corpus)

stop_words = nltk.corpus.stopwords.words('english')

tokens = wpt.tokenize(doc)

OUTPUT :
array (['Alex topper computer paper class 2016'], ['Tony, suspension rude behavior term'], ['Smiley
pneumonia term exam'], ['Ale ,born 19th january 1991 parents more'], ['best outgo student award
year'], ['Fail many subject'], ['transgender height 165cm'], ['Medicine leave submit pre-fix declare
holiday'])

```

Figure 4: Output of the pre-processed corpus

3.2 Feature Extraction

3.2.1 Term Frequency Count Vector using Uni-Gram model

The VSM ("Vector Space Model") is a simple mathematical model for the representation of unstructured data, particularly text, as numeric vectors in such a way that every dimension that belongs to the vector is viewed as a specific attribute or a feature. The Uni-gram type of model does the representation of all the documents into a numeric vector, where all the dimensions will be a definite word inside the corpus, and the value eventually is its number of occurrences in the document, which is denoted in a binary form (0 or 1).

```

# get all unique words in the corpus
vocab = cv.get_feature_names()

# show document feature vectors
pd.DataFrame(cv.matrix, columns=vocab)

```

Figure 5: Code snippet for term-frequency vectorizing -Unigram

The result in Figure 6 depicts that the documents have thus been converted into numeric vectors, where each document represents shows that the documents have been converted into numeric vectors such that each document is represented by one vector (row) in the above feature matrix.

	Alex	Topper	Computer	Paper	Class	Tony	Suspension	Rude	Behavior	...	Medicine	Leave	submit	declare	holiday	Pre- fix
0	1	1	1	1	1	0	0	0	0		0	0	0	0	0	0
1	0	0	0	0	0	1	1	1	1		0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0							
...																
7	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1

Figure 6: Term Frequency Count Vector

3.2.2 Weighted TF-IDF Model

The TF-IDF model exhibits the importance that a word has, i.e, the Term Frequency and the Inverse of the Document Frequency. The TF gives the total times that a given word appears in a document. The importance of a specific word $w(i)$ is represented as,

$$TF (Term) = \frac{Total\ Terms}{\sum_{n=0}^i total(Terms)_n} \quad (1)$$

In the above equation, the total (terms) represents the count of occurrences of the particular word in the document. The term in the denominator is the sum of the count of occurrences of all the words in the document. IDF is the measure of the ability of the word to distinguish among the categories. This can be obtained by the overall count of the documents, which can be contained in the document, divided by the log of the actual number of documents.

$$IDF (Term) = \log \frac{(Total\ docs)}{(Total\ no\ of\ docs, words)} + 0.01 \quad (2)$$

Total docs is the total count of the documents, and the denominator indicates the total count of the documents in which the particular word is present. The IDF measure is the count of documents that have the words fewer and which indicates the words have excellent class discrimination.

$$Weight (Term) = TF(Term) * IDF (term) \quad (3)$$

The weight is calculated by multiplying (1) and (2). The classical TF-IDF only considers the weight of the term frequency and does not take on the weights of the other features, and thus, the weighted TF-IDF is proposed. In any text corpus, the title and the initial paragraph play a vital role in determining the subject of the document. This will be helpful for the document classification and to restate the topic at the end of the document; therefore, these words, which at the start or end of the document can describe the topic more than other words of the document. Effective extraction of these words at the start or end of the document will have a great impact on the document's topic. If the context of each word is analyzed in detail, the algorithm's time efficiency will increase dramatically. The position of the first occurrence of a keyword and the position of the last occurrence of a keyword in a document are considered. The formula for the position of the first occurrence of a keyword can be described as:

$$F.P\ of\ word = \frac{F.P\ prior\ count\ (Word)+1}{\sum_{i=0}^n Count(word(i)- F.P\ prior\ count\ (Word))} \quad (4)$$

In equation (4), the meaning of FP Prior Count (word) is the number of all the words that appear before the position (not including this word) in the first appearance of the key word $\sum_{i=0}^n Count(word(i))$ is the total number of words in the document. The formula for the position of the last occurrence of a keyword can be described as:

$$L.P\ of\ word = \frac{L.P\ prior\ count\ (Word)+1}{\sum_{i=0}^n Count(word(i)- L.P\ prior\ count\ (Word))} \quad (5)$$

In equation (5), the meaning of LP prior Count (word) is the number of all the words that appear after the position (not including this word) in the last appearance of the key word. The denominator is the total number of words in the document. In summary, the formula for the adaptive weight of a key keyword's position can be described as:

$$Position\ Weight\ Word = \frac{1}{F.P\ (word)+L.P\ (word)} \quad (6)$$

Equation (6) is that the more front position of the key word which appears for the first time is, i.e. the value of First Position (word) smaller is, the more likely it is the word in the document title, abstract or first paragraph, and the more after position of the key word which appears for the last time is, i.e. the value of L Position (word) smaller is, the more likely it is the word in the document conclusion or last paragraph. Therefore, the formula for the weight of the keyword is optimized as:

$$Weight(Word, Doc) = \frac{TF * IDF * PW}{\sqrt{\sum_{word \in Doc} (TF * IDF * PW)^2}} \quad (7)$$

In equation (7), the TF is the occurrence frequency of the keyword, the IDF is the inverse document frequency of the keyword, and the Position Weight is the adaptive weight of the keyword's position.

3.2.3 Feature Vectors

The following is the code snippet applied to the Term Frequency Count vector to obtain the weighted feature vectors.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tv = TfidfVectorizer(min_df=0., max_df=1., use_idf=True)
tv_matrix = tv.fit_transform(norm_corpus)
tv_matrix = tv_matrix.toarray()
vocab = tv.get_feature_names()
pd.DataFrame(np.round(tv_matrix, 2), columns=vocab)
```

Figure 7: Code snippet for weighted feature vector generation

	Alex	Topper	Computer	Paper	Class	Tony	Suspension	Rude	Behavior	...	Medicine	Leave	submit	declare	holiday	Pre-fix
0	0.68	0.44	0.57	0.32	0.49	0	0	0	0		0	0	0	0	0	0
1	0	0	0	0	0	0.21	0.34	0.48	0.65		0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0
3	0	0	0	0	0.28	0.42	0	0.28	0		0	0	0	0.68	0.12	0.8
4	0	0.5	0	0.54	0.18	0.1	0	0	0		0.27	0.42	0	0	0.28	0.7
5	0.71	0	0	0.27	0.1	0.54	0.61	0	0		0.18	0.24	0.62	0.18	0.74	0.35
6	0.17	0	0	0	0	0.7	0.37	0.19	0.64		0	0	0	0.27	0.	0
7	0	0	0	0	0	0	0	0	0	0	0.33	0.48	0.62	0.18	0.52	0.64

Figure 8: Weighted Feature Vector representation

Figure 8 represents the weighted Feature vector representation of the corpus considered after applying the relative weights of the words, first position weight, and last position weights.

3.3 Clustering

3.3.1 Document Similarity

It is the process where the distance or the similarity is used as a metric, which is then used to identify how similar a certain corpus of text with any other document based on the features that are extracted from the documents. The cosine similarity is calculated as,

$$\cos x = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|} = \frac{\sum_1^n a_i b_i}{\sqrt{\sum_1^n a_i^2 \sum_1^n b_i^2}} \quad (8)$$

where $\vec{a} \cdot \vec{b}$ is the dot product of the given two vectors. The pairwise similarity is calculated by finding the document similarity between two corpora. Thus, if there are C number of documents in a given corpus, then the CXC matrix can be obtained such that all the rows and columns will represent the similarity score for a particular pair of documents. The cosine similarity is the most used, and it gives us the metric that represents the cosine of the angle in-between the vector notations of two documents. The smaller the cosine angle, the greater the similarity. Figure 9 shows the pair-wise similarity matrix of the corpus considered.

	0	1	2	3	4	5	6	7
0	1.000	0.000	0.000	0.121	0.000	0.511	0.000	0.000
1	0.000	1.000	0.000	0.000	0.792	0.000	0.000	0.000
2	0.000	0.000	1.000	0.000	0.000	0.000	0.000	0.621
3	0.000	0.000	0.000	1.000	0.000	0.000	0.529	0.000
4	0.265	0.185	0.268	0.158	0.657	0.786	0.621	0.641
5	0.000	0.481	0.101	0.354	0.268	0.157	0.725	0.631
6	0.128	0.264	0.000	0.267	0.741	0.364	0.185	0.369
7	0.327	0.254	0.542	0.000	0.154	0.000	0.265	0.741

Figure 9: Pair-wise Document Similarity matrix

From the figure, it is understood that the documents (0,5), (1,4), (2,7), and (3,6) are falling under similar categories.

3.3.2 Document Clustering

The process of clustering the documents helps to leverage the unsupervised learning into data points as a group. The Minimum variance method is used as the linkage criterion for the minimization of total variance within the cluster. Hence, in each step, the pair of clusters leading to the minimum increase in the total variance is calculated.

```
from scipy.cluster.hierarchy import dendrogram, linkage

Z = linkage(similarity_matrix, 'ward')

pd.DataFrame(Z, columns=['Document\Cluster 1', 'Document\Cluster 2', 'Distance', 'Cluster Size'], dtype='object')
```

Figure 10: Code Snippet for the DCM

Figure 10 shows the code snippet of the document clustering method imported for the corpus. The linkage matrix for the corpus obtained is as follows, and the figure shows that the features with only

low cluster sizes are extracted and in the corpus for our consideration, the feature set extracted is as $FS = \{Performance, Conduct, Absentism, \text{ and } Personal\}$.

Document	C1	C2	Distance	Cluster Size
0	2	7	0.2145	2
1	0	6	0.2852	2
2	5	8	0.3012	3
3	1	9	0.3421	3
4	3	4	2.8214	8
5	11	12	4.1254	9
6	10	13	6.4587	12

Figure 11: Linkage Matrix of the corpus

4 Experimental Setup and Metrics

The experiments are carried out under a minimum configuration. Python is used as the programming language, and Natural Language Processing toolkit 2.4 is used as the tool to perform pre-defined operations. The student text data from the US educational data hub was taken for the experiment that had 4605 text documents of 256Mb size.

4.1 Performance Metrics

The statistical parameters are used for error identification and to assess the overall performance of the proposed model. The various metrics used for the evaluation are as follows. Table 1 shows the basic parameters of evaluation.

Table 1: Fundamental Evaluation Metric

	Doesn't contain the target object or condition	Contain the target object or condition
Tests Negative or Accepted Null Condition	True Negative	False Negative
Tests Positive or Rejected Null Condition	False Positive	True Positive

4.1.1 True Positive

The true positive is when the final condition is marked as matching and correct, which shows the positive condition and denies the null hypothesis [20]. True positive is given with the symbol A. The true positive is given as follows:

$$TP = n_{11} = \text{number of such individuals} \quad (9)$$

4.1.2 True Negative

The true negative is when the final condition is marked as non-matching and correct, which shows the negative condition and accepts the null hypothesis. True negative is given with the symbol B. The true positive is given as follows:

$$TN = n_{00} = \text{number of such individuals} \quad (10)$$

4.1.3 False Positive

The false positive is when the final condition is marked as matching and incorrect, which shows the positive condition and denies the null hypothesis. A false positive is given with the symbol C. The false positive is given as follows:

$$FP = n_{01} = \text{number of such individuals} \quad (11)$$

4.1.4 False Negative

The false negative is when the final condition is marked as non-matching and incorrect, which shows the negative condition and accepts the null hypothesis. A false negative is given with the symbol D. The false negative is given as follows:

$$FN = n_{10} = \text{number of such individuals} \quad (12)$$

4.1.5 Recall

It is the test based on probability for accuracy that indicates the overall performance of the model in the presence of false-negative scenarios. This is calculated as,

$$\text{Recall} := \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (13)$$

4.1.6 Precision

It is the test based on probability for accuracy that indicates the overall performance of the model in the presence of positive scenarios. This is calculated as,

$$\text{Precision} := \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (14)$$

4.1.7 F1-Measure

It is the cumulative measure to assess the overall impact that the precision and the recall have in a case. The F-Measure is represented between 0 and 1 or from 0 to 100 and is normally determined based on the case and ranges of precision and recall. It is measured by,

$$F1 - \text{Measure} := 2 * \frac{(R * p)}{R + P} \quad (15)$$

Where R is recall, and p or P is precision.

4.1.8 Accuracy

The overall value for the accuracy is obtained by dividing the total number of true cases by all cases.

$$\text{Accuracy} := \frac{\text{True Positive} + \text{True Negative}}{\text{True Positive} + \text{True Negative} + \text{False Positive} + \text{False Negative}} \quad (16)$$

4.1.9 Logarithmic Loss

Cross-entropy loss, or in other words, the log loss, is a performance measure of a given classification whose output is the probability between 0 and 1. The value increases as the prediction value deviates from the original label. In the case of a multiclass classification, the log loss is calculated using,

$$\text{Log loss} = \sum_{c=1}^m y_{0,c} \log p_{0,c} \quad (17)$$

4.1.10 Mean Squared Error

Mean squared error is the mean of the squared difference in-between predictions and the actual observations. It is calculated as

$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}^2)}{n} \quad (18)$$

4.2 Results and Discussion

The results obtained through various experiments are tabulated and explained in the following section. The results obtained are compared with those of other state-of-the-art models such as TF-IDF (Term Frequency Inverse Document Frequency), Glove (Global Vectors for words representation), W2V (Word2vec), VSM (Vector Space Model), and BOW (Bag of Words).

Table 2: Comparative Analysis of Number of Features Extracted, Entropy, and MV

Method	Evaluation Measures		
	No Features Extracted	Entropy %	Minimum variance %
TF-IDF	51	56	57
GLove	62	76	74
W2V	74	71	62
VSM	82	74	70
BOW	95	64	81
WTF-IDF	36	39	52

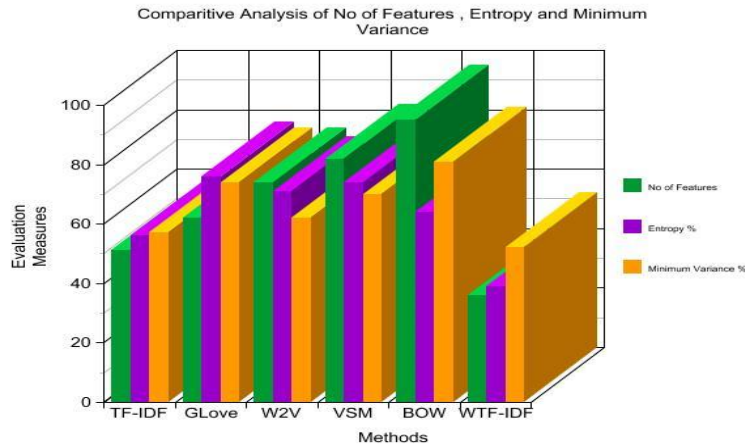


Figure 12: Comparative analysis of Number of Features, Entropy, and Minimum Variance

Table 2 shows the comparative analysis of the number of features extracted from the same documents, their entropy, and the minimum variance value obtained with that of the other models. It is observed from the table that the weighted TF-IDF model produces the less no of features when compared to other models. This is because the proposed method concentrates more accurately on the relative weights and thus leaves the redundant and unimportant features, unlike others, and it is evident from the low entropy and Minimum variance scores. Figure 12 depicts the same in a graphical manner.

Table 3 brings the comparative results of the most used performance metrics, namely Precision, Recall F-Measure. It is seen from the table that the proposed method has high precision and recall, and thus proves that the proposed method produces highly accurate results (high precision) and also returns a majority of positive results (high recall).

Table 3: Comparative analysis of precision, Recall, and F-score

Method	Performance Metrics		
	Precision	Recall	F-Measure
TF-IDF	0.89	0.81	0.84
GLove	0.81	0.76	0.72
W2V	0.76	0.71	0.69
VSM	0.79	0.74	0.70
BOW	0.68	0.64	0.59
WTF-IDF	0.94	0.96	0.98

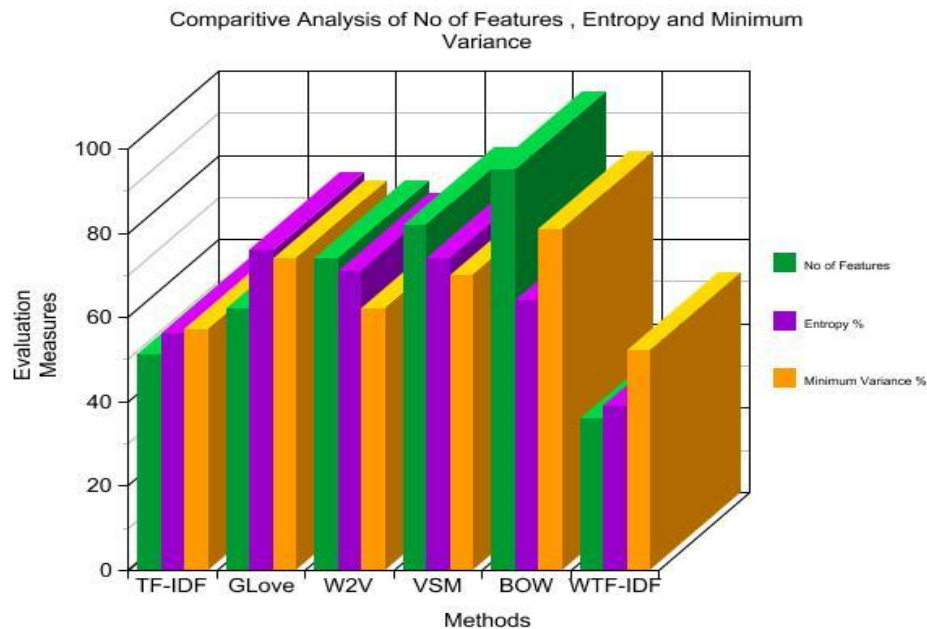


Figure 13: Comparison of Precision, Recall, and F-Measure

Table 4 gives the comparative results of the various models in terms of accuracy. It is seen that the proposed model has the highest accuracy of 94%. The same is depicted in Figure 14 as a graphical form.

Table 4: Comparative results of accuracy

Methods	Accuracy
TF-IDF	0.89
GLove	0.81
W2V	0.76
VSM	0.79
BOW	0.68
WTF-IDF	0.94

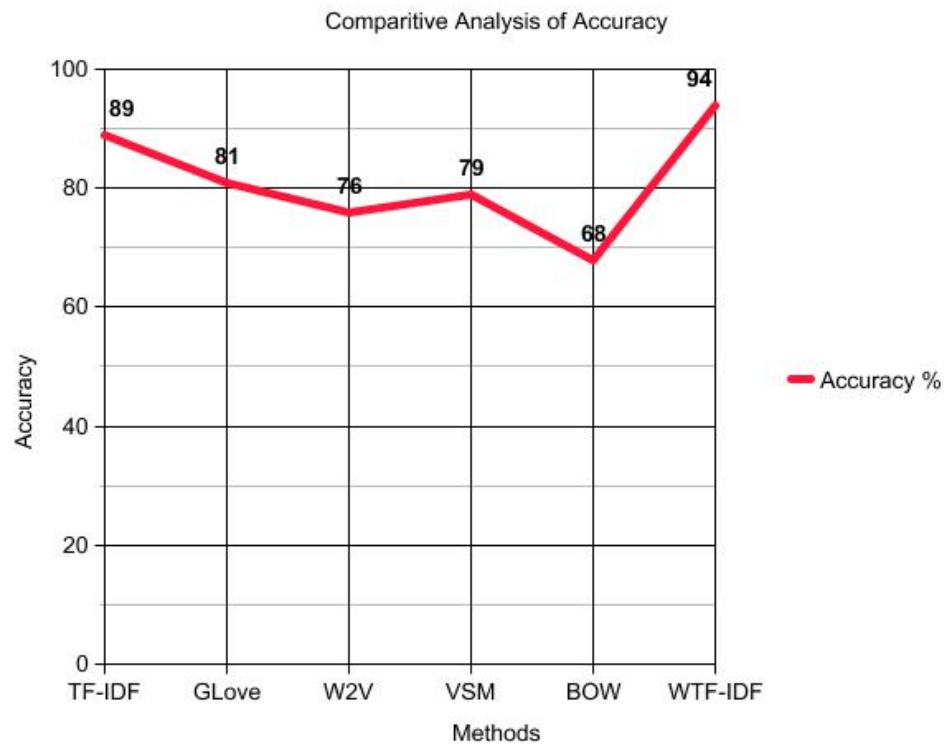


Figure 14: Comparison of Accuracy

Table 5: Comparison of Losses

Method	Performance Measures	
	Logarithmic Loss	Mean Squared Error
TF-IDF	0.26	0.21
GLove	0.41	0.36
W2V	0.37	0.51
VSM	0.59	0.44
BOW	0.61	0.64
WTF-IDF	0.14	0.11

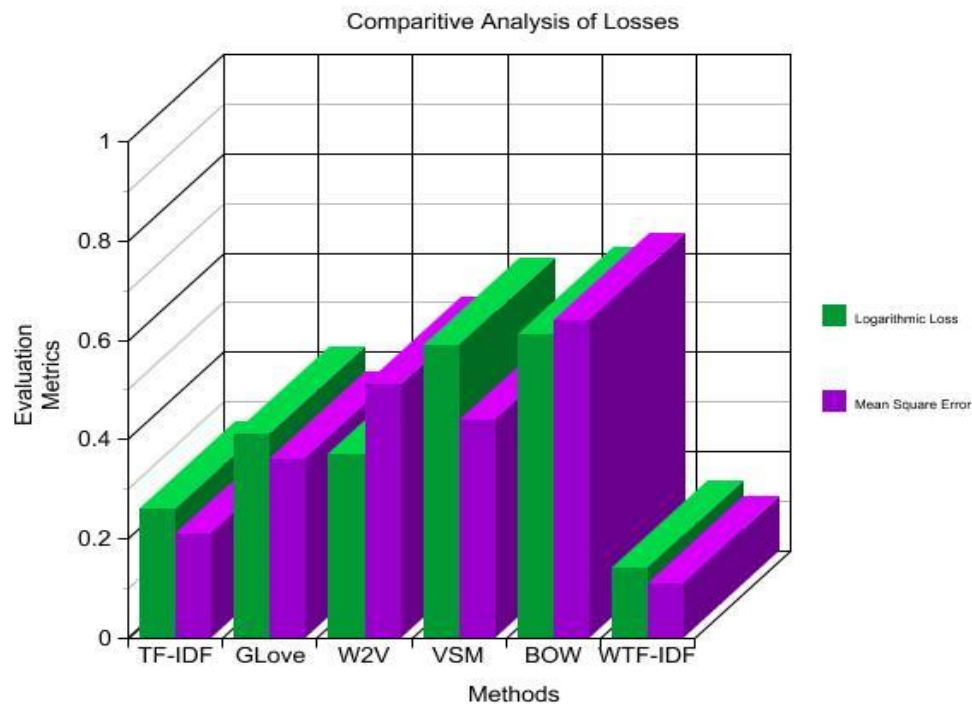


Figure 15: Comparison of Losses

Table 5 depicts the comparative results of the losses, namely the log loss and MSE. The same is represented in graphical form in Figure 15. It is obvious that the proposed methodology outperforms the other methods by having minimum losses. Ten performance metrics were chosen for the analysis of the results and the proposed methodology.

5 Conclusion

The process of Feature Extraction is used for the extraction of useful information from large unstructured data. The accumulation of data is happening at a very rapid rate because of technological advancements and modern storage facilities. Effective information extraction from these raw and big unstructured data is always tedious and challenging. Feature engineering is the best-known solution for reducing the dimensions of the Big-Data to be analyzed. Even though a handful of research studies have already been conducted in the domain, there are a few gaps that still exist. This paper proposes an automated Feature Extraction method based on the TF-IDF model using Skip-Gram from a text corpus. Further, document similarity features are calculated using the cosine similarity, and Document Clustering is done for grouping the similar features from the corpus together. Experimental analysis proves that the proposed methodology outperforms the other state-of-the-art embedded methods for text feature extraction. The research still has certain limitations, such as the sparsity in the features that are extracted, and also improvements are planned to be implemented so as to bring down the dimensionality so as to decrease the algorithm complexity and time complexity as well.

Acknowledgement: Not applicable.

Funding Statement: Not applicable.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, methodology and writing—original draft preparation: Yanglem Loiing Khomba; supervision and writing—review and editing: Khuman Iyyappan Moorthi. All authors reviewed the results and approved the final version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Gantz J, Reinsel D. The digital universe in 2020: Big data, bigger digital shadows, and biggest growth in the far east. IDC iView: IDC Analyze the future. 2012;2007(2012):1-6.
2. Wang Y, Kung L, Byrd TA. Big data analytics: Understanding its capabilities and potential benefits for healthcare organizations. *Technol Forecast Soc Change*. 2018;126:3-13.
3. Lomotey RK, Deters R. Topics and terms mining in unstructured data stores. In 2013 IEEE 16th international conference on computational science and engineering; 2013 Dec 3, Sydney, NSW, Australia; 2013. p. 854-861.
4. Lomotey RK, Deters R. RSenter: terms mining tool from unstructured data sources. *Int J Bu Process Integr Manag*. 2013;6(4):298-311.
5. Scheffer T, Decomain C, Wrobel S. Mining the web with active hidden Markov models. In *Proceedings 2001 IEEE International Conference on Data Mining*; 2001 Nov 29, San Jose, CA, USA; 2001. p. 645-646.
6. Lomotey RK, Jamal S, Deters R. SOPHRA: a mobile web services hosting infrastructure in mHealth. In *2012 IEEE first international conference on mobile services*; 2012 Jun 24, Honolulu, HI, USA; 2012. p. 88-95.
7. Ma Q, Tanaka K. Context-sensitive complementary information retrieval for text stream. In *International Conference on Database and Expert Systems Applications*; 2005 Aug 22, Berlin Heidelberg; 2005. p. 471-481.
8. Ren X, Zhou Y, Huang Z, Sun J, Yang X, Chen K. A novel text structure feature extractor for Chinese scene text detection and recognition. *IEEE Access*. 2017;5:3193-204.
9. Ren Y, Li D. Fast and Robust Wrapper Method for N -gram Feature Template Induction in Structured Prediction. *IEEE Access*. 2017;5:19897-908.
10. Jiang L, Zhang L, Li C, Wu J. A correlation-based feature weighting filter for naive Bayes. *IEEE Trans Knowl Data Eng*. 2018;31(2):201-13.
11. Chen J, Chen C, Liang Y. Optimized TF-IDF algorithm with the adaptive weight of position of word. In *2016 2nd International Conference on Artificial Intelligence and Industrial Engineering (AIIE 2016)*; 2016 Nov, 2016. p. 114-117.
12. Al-Talib GA, Hassan HS. A study on analysis of SMS classification using TF-IDF Weighting. *Int J Commun Netw Inf Secur*. 2013;1(5):189-94.
13. Singh A, Kumar D. Improved Feature Selection Forclassification Of Sentimental Reviews Using N-Gram Machine Learning Approach. *Glob j eng sci res*. 2018;5(7),278-288.
14. Zhu Z, Liang J, Li D, Yu H, Liu G. Hot topic detection based on a refined TF-IDF algorithm. *IEEE access*. 2019;7:26996-7007.
15. Yahav I, Shehory O, Schwartz D. Comments mining with TF-IDF: the inherent bias and its removal. *IEEE Trans Knowl Data Eng*. 2018;31(3):437-50.
16. Kim SW, Gil JM. Research paper classification systems based on TF-IDF and LDA schemes. *Hum-centric Comput Inf Sci*. 2019;9(1):30.
17. Li Y, Algarni A, Albathan M, Shen Y, Bijaksana MA. Relevance feature discovery for text mining. *IEEE Trans Knowl Data Eng*. 2014;27(6):1656-69.
18. Nayak AS, Kanive AP, Chandavekar N, Balasubramani R. Survey on pre-processing techniques for text mining. *Int J Eng Model*. 2016;5(6):16875-9.
19. Salloum SA, Al-Emran M, Monem AA, Shaalan K. Using text mining techniques for extracting information from research articles. In *Intelligent natural language processing: Trends and Applications*; 2017 Nov 18, 2017. p. 373-397.
20. Yang X, Lyu T, Lee CY, Bian J, Hogan WR, Wu Y. A study of deep learning methods for de-identification of clinical notes at cross institute settings. In *2019 IEEE International Conference on Healthcare Informatics (ICHI)*; 2019 Jun 10, Xi'an, China; 2019. p. 1-3.